LAB 11. CHUYỂN PROJECT EXPRESS MVC SANG REST API

1. Mục tiêu

Sau bài lab này, sinh viên có thể:

- Hiểu sự khác nhau giữa web render EJS và REST API.
- Tạo API trả dữ liệu JSON bằng Express.
- Tái sử dụng **`models`** đã có từ Lab 10.
- Tạo route API riêng trong thư mục **`routes`**.
- Tạo controller API riêng trong thư mục **`controllers`**.
- Xây dựng API CRUD cho bảng **`posts`**.
- Chuẩn bị backend để frontend React có thể gọi API.

2. Yêu cầu trước khi thực hiện

Sinh viên đã hoàn thành Lab 10 Express MVC.

Trong Lab 11, chúng ta không xóa phần EJS cũ mà chỉ bổ sung thêm REST API để React có thể kết nối sau này.

3. Ý tưởng chuyển sang REST API

Ở Lab 10, route đang render giao diện EJS:

```
res.render("posts/index", { posts })
```

Trong REST API, server không render HTML nữa mà trả về JSON:

```
res.json(posts)
```

PHẦN 1. CHUẨN BỊ PROJECT

Lab 11.1. Copy project Lab 10

Copy project express-mvc Lab 10 và đổi tên thành:

```
express-api
```

Chạy thử:

```
node app.js
```

Truy cập:

```
http://localhost:3000/news
```

Nếu project vẫn chạy bình thường thì bắt đầu bổ sung API.

Lab 11.2. Tạo thư mục API

Trong project, tạo thêm:

controllers/api
routes/api

Cấu trúc sau khi tạo:

```
express-api/
├── controllers/
│   ├── postController.js
│   ├── authController.js
│   └── api/
├── routes/
│   ├── postRoute.js
│   ├── authRoute.js
│   └── api/
```

PHẦN 2. API LẤY DANH SÁCH BÀI VIẾT

Lab 11.3. Tạo API Controller cho posts

Tạo file:

controllers/api/postApiController.js

Thêm nội dung ban đầu:

```
const postModel = require("../../models/postModel")
async function index(req, res) {
  try {
    const posts = await postModel.getAllPosts()

    res.json({
      success: true,
      data: posts
    })
  } catch (error) {
    console.log(error)
```

```
    res.status(500).json({
      success: false,
      message: "Lỗi khi lấy danh sách bài viết"
    })
  }
}

module.exports = {
  index
}
```

Lab 11.4. Tạo API Route cho posts

Tạo file:

routes/api/postApiRoute.js

Nội dung:

```
const express = require("express")
const router = express.Router()
const postApiController =
require("../../controllers/api/postApiController")
router.get("/", postApiController.index)
module.exports = router
```

Lab 11.5. Gắn API route vào app.js

Trong **app.js**, import route API:

```
const postApiRoutes = require("./routes/api/postApiRoute")
```

Thêm route /**api/posts**:

```
app.use("/api/posts", postApiRoutes)
```

Lab 11.6. Kiểm tra API danh sách bài viết

Chạy server:
node app.js
Truy cập trình duyệt hoặc Postman:
http://localhost:3000/api/posts

Kiểm tra bằng Postman và chụp ảnh kết quả vào đây

PHẦN 3. API LẤY CHI TIẾT BÀI VIẾT

Lab 11.7. Tạo hàm show trong API Controller

Mở file:

controllers/api/postApiController.js

Thêm hàm:

```
async function show(req, res) {
  try {
    const id = req.params.id

    const post = await postModel.getPostById(id)

    if (!post) {
      return res.status(404).json({
        success: false,
        message: "Không tìm thấy bài viết"
      })
    }

    res.json({
      success: true,
      data: post
    })
  } catch (error) {
    console.log(error)

    res.status(500).json({
      success: false,
      message: "Lỗi khi lấy chi tiết bài viết"
    })
  }
}
```

Sửa `module.exports`:

```
module.exports = {
  index, show
}
```

Lab 11.8. Thêm route chi tiết

Mở file:

routes/api/postApiRoute.js

Thêm route **/:id** :

```
const express = require("express")

const router = express.Router()

const postApiController =
require("../../controllers/api/postApiController")

router.get("/", postApiController.index)

router.get("/:id", postApiController.show)

module.exports = router
```

Lab 11.9. Kiểm tra API chi tiết

Truy cập:

```
http://localhost:3000/api/posts/1
http://localhost:3000/api/posts/6
```

Kiểm tra bằng Postman và chụp ảnh kết quả vào đây

PHẦN 4. API TÌM KIẾM BÀI VIẾT

Lab 11.10. Thêm hàm search

Trong controllers/api/postApiController.js, thêm:

```
async function search(req, res) {
  try {
    const keyword = req.query.keyword || ""

    const posts = await postModel.searchPosts(keyword)

    res.json({
      success: true,
      keyword: keyword,
      data: posts
    })
  } catch (error) {
```

```
    console.log(error)

    res.status(500).json({
      success: false,
      message: "Lỗi khi tìm kiếm bài viết"
    })
  }
}
```

Sửa `module.exports`:

```
module.exports = {
  index,
  show,
  search
}
```

Lab 11.11. Thêm route tìm kiếm

Trong `routes/api/postApiRoute.js`, thêm route tìm kiếm trước `/:id`:

```
const express = require("express")

const router = express.Router()

const postApiController =
require("../../controllers/api/postApiController")

router.get("/", postApiController.index)
router.get("/search", postApiController.search)
router.get("/:id", postApiController.show)

module.exports = router
```

Lưu ý:

Route `/search` phải đặt trước `/:id`.

Lab 11.12. Kiểm tra API tìm kiếm

Truy cập:

http://localhost:3000/api/posts/search?keyword=node

Kiểm tra bằng Postman và chụp ảnh kết quả vào đây

PHẦN 5. API THÊM BÀI VIẾT

Lab 11.13. Cấu hình Express đọc JSON body

Trong `app.js`, thêm middleware express.json dưới dòng:

```
app.use(express.urlencoded({ extended: true }))
app.use(express.json())
```

Ý nghĩa:

```
express.urlencoded() → đọc dữ liệu từ form HTML
express.json()       → đọc dữ liệu JSON gửi từ React/Postman
```

Lab 11.14. Thêm hàm store

Trong **`controllers/api/postApiController.js`**, thêm:

```
async function store(req, res) {
  try {
    const title = req.body.title
    const description = req.body.description

    if (!title || !description) {
      return res.status(400).json({
        success: false,
        message: "Vui lòng nhập đầy đủ title và description"
      })
    }

    await postModel.createPost(title, description)

    res.status(201).json({
      success: true,
      message: "Thêm bài viết thành công"
    })
  } catch (error) {
    console.log(error)

    res.status(500).json({
      success: false,
      message: "Lỗi khi thêm bài viết"
    })
  }
}

module.exports = {
```

```
  index,
  show,
  search,
  store
}
```

Lab 11.15. Thêm route POST

Trong `routes/api/postApiRoute.js`, thêm route post:

```
const express = require("express")

const router = express.Router()

const postApiController =
require("../../controllers/api/postApiController")

router.get("/", postApiController.index)

router.get("/search", postApiController.search)

router.get("/:id", postApiController.show)

router.post("/", postApiController.store)

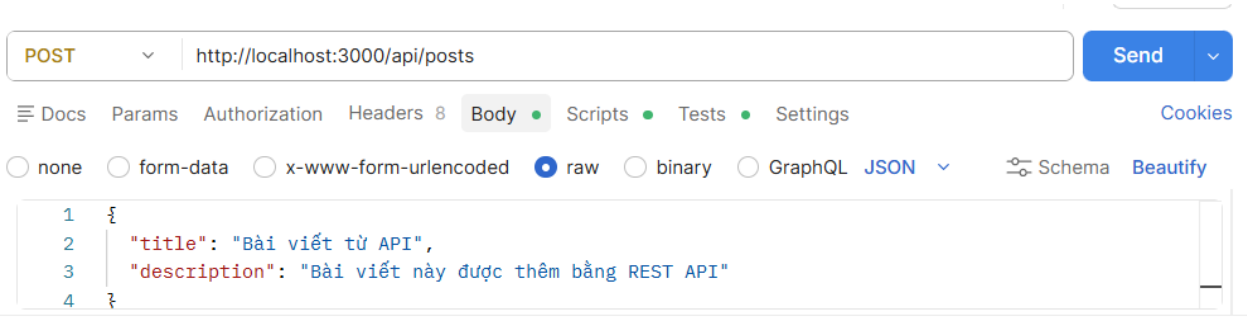
module.exports = router
```

Lab 11.16. Kiểm tra API thêm bằng Postman

Mở Postman.
Chọn method: `POST`
URL: `http://localhost:3000/api/posts`
Tab Body:
`raw → JSON`

Nhập:

```
{
  "title": "Bài viết từ API",
  "description": "Bài viết này được thêm bằng REST API"
}
```



Kiểm tra bằng Postman và chụp ảnh kết quả vào đây

Sau đó kiểm tra lại:

GET http://localhost:3000/api/posts

Kiểm tra bằng Postman và chụp ảnh kết quả vào đây

PHẦN 6. API CẬP NHẬT BÀI VIẾT

Lab 11.17. Thêm hàm update

Trong `controllers/api/postApiController.js`, thêm:

```
async function update(req, res) {
  try {
    const id = req.params.id
    const title = req.body.title
    const description = req.body.description

    const post = await postModel.getPostById(id)

    if (!post) {
      return res.status(404).json({
        success: false,
        message: "Không tìm thấy bài viết"
      })
    }

    if (!title || !description) {
      return res.status(400).json({
        success: false,
        message: "Vui lòng nhập đầy đủ title và description"
      })
    }

    await postModel.updatePost(id, title, description)
```

```
    res.json({
      success: true,
      message: "Cập nhật bài viết thành công"
    })
  } catch (error) {
    console.log(error)

    res.status(500).json({
      success: false,
      message: "Lỗi khi cập nhật bài viết"
    })
  }
}
module.exports = {
  index,
  show,
  search,
  store,
  update
}
```

Lab 11.18. Thêm route PUT

Trong `routes/api/postApiRoute.js`, thêm route put:

```
const express = require("express")
const router = express.Router()

const postApiController =
require("../../controllers/api/postApiController")
router.get("/", postApiController.index)
router.get("/search", postApiController.search)
router.get("/:id", postApiController.show)
router.post("/", postApiController.store)
router.put("/:id", postApiController.update)

module.exports = router
```

Lab 11.19. Kiểm tra API cập nhật bằng Postman

Method: `PUT`

URL: `http://localhost:3000/api/posts/1`

Body:

```
{
  "title": "Tiêu đề đã cập nhật",
  "description": "Mô tả đã cập nhật bằng REST API"
}
```

Kiểm tra bằng Postman và chụp ảnh kết quả vào đây

PHẦN 7. API XÓA BÀI VIẾT

Lab 11.20. Thêm hàm destroy

Trong `controllers/api/postApiController.js`, thêm hàm destroy:

```
async function destroy(req, res) {
  try {
    const id = req.params.id

    const post = await postModel.getPostById(id)

    if (!post) {
      return res.status(404).json({
        success: false,
        message: "Không tìm thấy bài viết"
      })
    }

    await postModel.deletePost(id)

    res.json({
      success: true,
      message: "Xóa bài viết thành công"
    })
  } catch (error) {
    console.log(error)

    res.status(500).json({
      success: false,
      message: "Lỗi khi xóa bài viết"
    })
  }
}
module.exports = {
  index,
  show,
  search,
  store,
  update,
```

```
  destroy
}
```

Lab 11.21. Thêm route DELETE

Trong `routes/api/postApiRoute.js`, thêm:

```
const express = require("express")

const router = express.Router()

const postApiController =
require("../../controllers/api/postApiController")

router.get("/", postApiController.index)
router.get("/search", postApiController.search)
router.get("/:id", postApiController.show)
router.post("/", postApiController.store)
router.put("/:id", postApiController.update)
router.delete("/:id", postApiController.destroy)

module.exports = router
```

Lab 11.22. Kiểm tra API xóa bằng Postman

Method: `DELETE`

URL: `http://localhost:3000/api/posts/1`

Kiểm tra bằng Postman và chụp ảnh kết quả vào đây

PHẦN 8. TẠO API ĐĂNG NHẬP

Lab 11.23. Tạo Auth API Controller

Tạo file:

`controllers/api/authApiController.js`

Nội dung:

```
const userModel = require("../../models/userModel")
```

```
async function login(req, res) {
  try {
    const username = req.body.username
    const password = req.body.password

    const user = await
userModel.findUserByUsernameAndPassword(username, password)

    if (!user) {
      return res.status(401).json({
        success: false,
        message: "Sai username hoặc password"
      })
    }

    req.session.user = {
      id: user.id,
      username: user.username,
      fullname: user.fullname
    }

    res.json({
      success: true,
      message: "Đăng nhập thành công",
      user: req.session.user
    })
  } catch (error) {
    console.log(error)

    res.status(500).json({
      success: false,
      message: "Lỗi khi đăng nhập"
    })
  }
}

function logout(req, res) {
  req.session.destroy(() => {
    res.json({
      success: true,
      message: "Đăng xuất thành công"
    })
  })
}

function me(req, res) {
```

```
  if (!req.session.user) {
    return res.status(401).json({
      success: false,
      message: "Chưa đăng nhập"
    })
  }

  res.json({
    success: true,
    user: req.session.user
  })
}

module.exports = {
  login,
  logout,
  me
}
```

Lab 11.24. Tạo Auth API Route

Tạo file:

routes/api/authApiRoute.js

Nội dung:

```
const express = require("express")

const router = express.Router()

const authApiController =
require("../../controllers/api/authApiController")

router.post("/login", authApiController.login)

router.post("/logout", authApiController.logout)

router.get("/me", authApiController.me)

module.exports = router
```

Lab 11.25. Gắn Auth API route vào app.js

Trong `app.js`, import:

```
const authApiRoutes = require("./routes/api/authApiRoute")
```

Thêm dòng:

```
app.use("/api/auth", authApiRoutes)
```

Lab 11.26. Kiểm tra API đăng nhập

Method:

```
POST
```

URL:

```
http://localhost:3000/api/auth/login
```

Body:

```
{
  "username": "admin",
  "password": "123456"
}
```

Kiểm tra bằng Postman và chụp ảnh kết quả vào đây

Kiểm tra trạng thái đăng nhập:

```
GET http://localhost:3000/api/auth/me
```

Kiểm tra bằng Postman và chụp ảnh kết quả vào đây

Đăng xuất:

```
POST http://localhost:3000/api/auth/logout
```

Kiểm tra bằng Postman và chụp ảnh kết quả vào đây

PHẦN 9. BẢO VỆ API THÊM, SỬA, XÓA

Lab 11.27. Tạo middleware riêng cho API

Mở file:

middlewares/authMiddleware.js

Thêm hàm:

```
function requireApiLogin(req, res, next) {
  if (!req.session.user) {
    return res.status(401).json({
      success: false,
      message: "Bạn cần đăng nhập để thực hiện chức năng này"
    })
  }

  next()
}

module.exports = {
  requireLogin,
  usesession,
  requireApiLogin
}
```

Lab 11.28. Dùng lại middleware requireLogin

Trong `routes/api/postApiRoute.js`, import middleware:

```
const express = require("express")

const router = express.Router()

const postApiController =
require("../../controllers/api/postApiController")

const {
  requireApiLogin
} = require("../../middlewares/authMiddleware")

router.get("/", postApiController.index)

router.get("/search", postApiController.search)

router.get("/:id", postApiController.show)
```

```
router.post("/", requireApiLogin, postApiController.store)

router.put("/:id", requireApiLogin, postApiController.update)

router.delete("/:id", requireApiLogin,
postApiController.destroy)

module.exports = router
```

Lab 11.29. Kiểm tra bảo vệ API

Khi chưa đăng nhập, gọi:

```
POST http://localhost:3000/api/posts
```

Kiểm tra bằng Postman và chụp ảnh kết quả vào đây

Sau khi gọi API đăng nhập:

```
POST http://localhost:3000/api/auth/login
```

Gọi lại:

```
POST http://localhost:3000/api/posts
```

Kiểm tra bằng Postman và chụp ảnh kết quả vào đây